

# MAGEAM — Inventario técnico de funciones IA

> Solicitado por Gonzalo (Producto). Inventario de cada lugar donde MAGEAM invoca a Claude (Anthropic API). Útil para auditar qué funciona con LLM real, qué con algoritmo determinístico, y qué prompts mandamos.

>

> **Fecha:** 2026-04-30 · **Modelo activo:** `claude-sonnet-4-6` · **API key:** `ANTHROPIC_API_KEY` env var en VPS (seteado).

## 1. Resumen ejecutivo

| Capa                  | Funciones IA reales                                             | Algoritmo (sin LLM)                      | Total cards |
|-----------------------|-----------------------------------------------------------------|------------------------------------------|-------------|
| 1 — Captura           | #1 Vision · #2 Voice · #4 Classification                        | #3 Duplicate detect (algoritmo)          | 4           |
| 2 — Validación + Plan | #6 Smart Assignment · #7 Auto-Level NL parser                   | #5, #8, #9 (reglas SAP)                  | 5           |
| 3 — Ejecución         | —                                                               | #10, #11, #12 (lógica determinística)    | 3           |
| 4 — Análisis          | #14 Crónicas · #18 NLP causas · #19 Stock OC · #20 Cost element | #13, #15, #16, #17 (algoritmos clásicos) | 8           |
| 5 — Cierre            | —                                                               | #21, #22 (cierre del ciclo)              | 2           |
| **Total**             | **9 endpoints Claude reales**                                   | **13 algoritmos**                        | **22**      |

## 2. Endpoints con LLM (Claude real)

Todos usan `claude-sonnet-4-6` , `max_tokens` entre 400-2000 según caso. Si `ANTHROPIC_API_KEY` está vacío, hay fallback determinístico (heurística keyword) — degrada bien.

### #1 — Vision AI (foto → equipo + falla)

- **Endpoint:** `POST /api/v1/captures/photo`
- **Servicio:** `api/services/vision_service.py`
- **Modelo:** Claude vision (multimodal)
- **Input:** imagen base64
- **Output:** `{equipment_tag_detected, failure_mode_detected, confidence}`
- **Status:** ■ FUNCIONAL · usado en `/failure-capture` botón cámara.

### #2 — Voice capture (dictado → WR estructurado)

- **Endpoint:** `POST /api/v1/agentiv/voice-capture`
- **Servicio:** `api/services/agentiv_voice_capture_service.py:_classify_with_claude`

- **Input:** texto raw transcripto
- **Output:** dict con `failure_category`, `failure_object_part`, `failure_symptom`, `failure_cause`, `suggested_action`, `priority`, `priority_reason`, `activity_class`, `work_conditions`
- **Prompt:** catálogo SAP estructurado (MECANICO/ELECTRICO/INSTRUMENTACION × partes/síntomas/causas) — Claude elige exactamente UN valor de cada lista.
- **Status:** ■ FUNCIONAL · botón rojo "grabar" en `/failure-capture` .

#### #4 — AI Classification del form manual

- **Endpoint:** `POST /api/v1/work-requests/manual` (llama internamente a `_classify_with_claude` )
- **Cuándo:** cada vez que el usuario submite el form Failure Capture con un `problem_description` no vacío.
- **Bump de prioridad:** si Claude sugiere prioridad MAYOR que la del usuario (ej: usuario marca P3 pero la descripción menciona "humo + emergencia"), la levanta a P1/P2 y deja flag `priority_bumped_by_ai=true` + `ai_priority_reason` para audit.
- **Output persistido en `ai_classification`:** `priority_suggested`, `priority_user`, `priority_bumped_by_ai`, `failure_type`, `failure_class`, `failure_category`, `failure_object_part`, `ai_suggested_action`, `ai_work_conditions`, `ai_priority_reason`, `source: 'manual_form+claude'` .
- **Status:** ■ FUNCIONAL desde 2026-04-30 (commit ee558b9 ). Antes era stub.

#### #6 — Smart Assignment con razonamiento

- **Endpoint:** `POST /api/v1/assignments/rank-for-operation`
- **Pipeline:** scoring algorítmico (40 specialty + 30 skill + 20 shift + HH) ranquea top-10 → Claude analiza top-3 y devuelve recomendación + reasoning + warnings.
- **Función Claude:** `api/routers/assignments.py:_claude_recommend_technician`
- **Output:** `{candidates: [...], ai_recommendation: {recommended_worker_id, reasoning, warnings: []}}`
- **Caso uso:** cuando 3 técnicos tienen score idéntico, Claude desempata por skills críticos contra la descripción del WO.
- **Status:** ■ FUNCIONAL · `/scheduling` modal "■ IA".

#### #7 — Auto-Level NL parser

- **Endpoint:** `POST /api/v1/scheduling/parse-autolevel-instructions`
- **Input:** texto NL del usuario ("lunes liviano por reunión, no programes el viernes 2 mayo, prioriza chancador") + lista de WOs + días de la semana
- **Output:** constraints estructurados — `{priority_boost_wos, priority_boost_equipment, deprioritize_wos, light_days, excluded_days, capacity_override_pct, include_weekend, summary}`
- **Mejora sobre keywords:** mapea fechas relativas ("lunes" → 2026-04-28 ), entiende equipos por tag ("chancador CRU-CON-HP-01"), excluye feriados.

- **Status:** ■ FUNCIONAL · botón "■ Interpretar con Claude" en `/scheduling` Auto-Level wizard.

## #14 — Análisis crónicas con causa raíz

- **Endpoint:** `POST /api/v1/analytics/chronic-failures-analyze`
- **Input:** clusters detectados algorítmicamente ( $\geq 3$  reps en 7d) + sample descriptions
- **Output:** para cada cluster `{root_cause_hypothesis, recommended_action, confidence: high|medium|low}`
- **Caso uso:** el detector encuentra "PMP-AGUA-01: vibración axial 4 veces" → Claude propone "rodamiento NDE deteriorado, reemplazar + verificar juego axial".
- **Status:** ■ FUNCIONAL · panel morado en sección "Fallas Crónicas" de Performance Analysis.

## #18 — NLP causas no-cumplimiento

- **Endpoint:** `POST /api/v1/analytics/classify-noncompliance`
- **Input:** lista de notas raw ( `execution_notes` , `closure_notes` , `notif_notes` )
- **Clasifica en 7 categorías estándar Jorge:** REPUESTO\_FALTANTE, OPERACIONES\_NO\_LIBERO, SERVICIO\_EXTERNO\_NO\_LLEGO, HERRAMIENTA\_FALTANTE, EQUIPO\_APOYO\_NO\_DISPONIBLE, VENTANA\_INSUFICIENTE, SEGURIDAD\_LOTO.
- **Detecta categorías emergentes:** ej: "PERSONAL\_INSUFICIENTE" si una nota dice "no había soldador certificado".
- **Output:** `{categories: [{category, count, samples: [...]}], emerging: [...], total_notes, ai_used}`
- **Antes era:** 7 regex hardcoded ( `/repuesto.*no\s+(corresp|disponible|llegó)/i` ).
- **Status:** ■ FUNCIONAL desde 2026-04-30 ( `99559e6` ).

## #19 — Stock OC recommend con justificación

- **Endpoint:** `POST /api/v1/analytics/stock-oc-recommend`
- **Input:** forecast determinístico (consumo 90d + demanda 60d + stock disponible).
- **Filtra:** códigos en riesgo (cobertura  $< 30$ d o stockout proyectado).
- **Output:** para cada código crítico `{qty_suggested, priority: HIGH|MEDIUM|LOW, reasoning: "Cobertura 2.5d (<7d); demanda 60d=10 unidades. Sugiero 8 uds..."}`
- **Status:** ■ FUNCIONAL · panel morado en sección Stock Forecast de PA.

## #20 — Cost element classifier

- **Servicio:**  
`api/services/cost_analysis_service.py:_classify_materials_with_claude`
- **Cuándo:** al computar `cost_analysis` — si un material tiene `description` pero le falta `cost_element` , batch-clasifica con Claude.
- **Categorías SAP:** REPUESTO\_CONSUMIBLE, REPUESTO\_CRITICO, REPUESTO\_ELECTRICO, INSUMO\_LUBRICANTE, HERRAMIENTA\_EQUIPO, MANO\_DE\_OBRA, SERVICIO\_EXTERNO.
- **Cache:** `_COST_ELEMENT_CACHE` en memoria global (no re-clasifica descripciones repetidas).

- **Antes era:** `m.get("cost_element")` or `"REPUESTO_CONSUMIBLE"` (default ciego).
- **Status:** ■ FUNCIONAL desde 2026-04-30.

### 3. Capacidades agénticas SIN LLM (algoritmo)

Cards #3, #5, #8, #9, #10, #11, #12, #13, #15, #16, #17, #21, #22 — todas funcionales pero **sin invocar Claude** porque la lógica es determinística:

- **#3** Duplicate detection: jaccard + sequence ratio + time-decay.
- **#5** Express PM03: regla SAP, no necesita IA.
- **#8** Bloqueo PM01/02 → falla: `HTTPException 409` por regla.
- **#9** Cancelación con tipología: dropdown enum + linkeo de OT absorbente.
- **#10** Notif parcial: agrupa por shift + technician + WS broadcast.
- **#11** Pre-close gates: 5 reglas (ALL\_OPS\_DONE, HH\_VARIANCE\_OK, MATERIALS\_OK, SUPERVISOR\_QA, NO\_OPEN\_NOTIFS).
- **#12** Stock auto-decrement: hook idempotente al cierre.
- **#13** Bad Actors: cruce con criticality threshold  $\geq 2 / \geq 3$ .
- **#15** Auto-RCA trigger: scan + 5W2H pre-fill template.
- **#16** Retrabajos <24h: correlación temporal.
- **#17** Pareto + Jack-Knife: fórmulas Jorge (validadas 1:1 contra Excels).
- **#21** FMECA push auto: idempotente por `source_ref` .
- **#22** Close-the-loop tracking: cuenta `improvement_actions` por status.

Estas no se "convierten en IA" — la lógica es transparente, auditable, sin riesgo de alucinación. **No se exagera el label:** las cards no dicen "IA" si no la usan.

### 4. Modelo de decisión: ¿cuándo Claude, cuándo no?

| Criterio                                                    | Usar Claude                                  | Usar algoritmo             |
|-------------------------------------------------------------|----------------------------------------------|----------------------------|
| Texto libre del usuario                                     | ■ NLP es necesario                           | —                          |
| Catálogo cerrado (P1/P2/P3, MEC/ELEC)                       | ■ si hay que mapear desde NL                 | ■ si ya viene estructurado |
| Decisión que requiere razonamiento ("por qué este técnico") | ■ output con reasoning                       | —                          |
| Cálculo numérico determinístico (HH, %, suma)               | —                                            | ■ siempre                  |
| Auditoría regulatoria (no se puede alucinar)                | —                                            | ■ siempre                  |
| Ranking + interpretación                                    | ■ híbrido (algoritmo rankea, Claude explica) | —                          |

## 5. Configuración VPS

- **Variable:** `ANTHROPIC_API_KEY` (env del container `ocp-backend` ).
- **Modelo:** `claude-sonnet-4-6` hardcodedo en cada llamada.
- **Sin streaming:** todas las llamadas son `client.messages.create(...)` sync.
- **Latencia típica:** 2-7s por llamada (Smart Assignment ~6s, Classify ~3s).
- **Sin retry** automático — si Claude falla, fallback determinístico.
- **Sin rate limiting custom** — confiamos en el rate limit de Anthropic.

## 6. Costos estimados (orden de magnitud)

Por llamada Claude Sonnet 4.6 (~\$3 in / \$15 out per 1M tokens):

| Función              | Input típico                | Output típico    | Costo/llamada |
|----------------------|-----------------------------|------------------|---------------|
| #4 AI Classification | ~500 tokens prompt          | ~200 tokens JSON | ~\$0.005      |
| #6 Smart Assignment  | ~600 tokens (3 candidatos)  | ~150 tokens      | ~\$0.004      |
| #7 Auto-Level parser | ~800 tokens (50 WOs)        | ~250 tokens      | ~\$0.006      |
| #14 Crónicas analyze | ~700 tokens (5 clusters)    | ~400 tokens      | ~\$0.008      |
| #18 NLP causas       | ~2000 tokens (80 notas)     | ~600 tokens      | ~\$0.015      |
| #19 OC recommend     | ~1500 tokens (15 códigos)   | ~500 tokens      | ~\$0.012      |
| #20 Cost element     | ~1500 tokens (60 materials) | ~500 tokens      | ~\$0.012      |

**Estimación uso plant Goldfields:** ~50 llamadas/día → **~\$0.5/día** = ~\$15/mes per plant.

Si quisiéramos cachear más agresivo (compartir resultados entre WRs similares), bajaría a ~\$5-8/mes.

## 7. Próximas funciones IA (Capa 6 Phase 2 — pendientes)

De los 20 ítems en Capa 6 ( `/agentic-capabilities` ), los que requieren Claude real son:

- #33 RCM Strategy Advisor (analizar histórico Weibull + falla → propone cambio frecuencia)
- #34 Shift Handover Assistant (consolidar turno saliente para entrante)
- #35 Post-Maint Learning (embeddings + retrieval para troubleshooting)
- #40 Knowledge Base Curator (auto-tag + búsqueda semántica)

Los demás Phase 2 son determinísticos (conector Jigsaw, KPI Watchdog, dashboards HSE, etc.).

## 8. Archivos clave

- Backend: `api/services/agentive_voice_capture_service.py` ,  
`api/routers/analytics.py` , `api/routers/assignments.py` ,  
`api/routers/scheduling.py` , `api/routers/work_requests.py` ,  
`api/services/cost_analysis_service.py`
  - Frontend: `frontend/src/pages/PerformanceAnalysis.jsx` ,  
`frontend/src/pages/Scheduling.jsx` ,  
`frontend/src/components/SmartAssignModal.jsx`
  - Docs relacionados: `docs/AGENTIC_CAPABILITIES.md` ,  
`docs/PRESENTACION_FUNCIONES_AGENTICAS.md` , `docs/AGENTIC_SOLUTIONS_ROADMAP.md`
- 

*Documento vivo. Actualizo con feedback de Gonzalo.*